SYSTEMS ENGINEERING WHITEPAPER & REPRODUCIBLE WORKSTATION CASE STUDY

# Beyond VRAM Capacity: A Single-Workstation Case Study of Strata, Qwen3.8-Flash-Next, and OpenCode on an RTX 5080

**Author:** Arun C Michael | **Date:** November 2026 | **Subject:** OCF V1 Single-Slot Local Coding Orchestration, Hybrid Memory Placement & Compaction Control

## Abstract

A persistent misconception in local language model deployment is that a GPU's physical video memory (VRAM) ceiling defines the maximum total parameter weight of a usable coding model. On a 16 GB consumer GPU, this assumption restricts practitioners to small dense models that frequently struggle with multi-file reasoning or complex algorithmic synthesis. This paper documents a single-workstation systems engineering architecture conducted by Arun C Michael **[4]** that runs an 83.62 GB (decimal on-disk weight) quantized sparse model— `ISTA-DASLab/Qwen3.8-Flash-Next-GSQ-RCO-GGUF` **[2]** ( `IQ3_S` quantization, referred to operationally as `FlashNext` )—on an NVIDIA RTX 5080 (16,303 MiB reported VRAM) paired with a high-RAM Windows host and local NVMe storage.

Rather than claiming a universal model benchmark or unconditional parity with frontier cloud APIs, this study details the systems engineering required to make an oversized sparse model operationally viable for autonomous software engineering tasks. We present the **OpenCode FlashNext (OCF V1)** orchestration protocol, which couples an external IDE orchestrator (such as Cursor, Claude Code, or Windsurf) with an isolated **OpenCode** client **[3]** and a loopback **Strata** inference endpoint **[1]** across a serialized, single-slot First-In-First-Out (FIFO) job queue. We report verified empirical measurements across five core engineering interventions: (1) explicit Per-Layer Embedding / lookup ( `PLE` ) placement on direct NVMe storage to eliminate an automatic 28.8 GB (26.8 GiB) host-RAM residency spike; (2) client context budgeting at 131,072 tokens with a hard 81,920-token compaction trigger; (3) wire-verified disabling of inherited reasoning tokens during context compaction, reducing summary generation time from 69.282 seconds to 18.360 seconds (a 69.7% token reduction on a paired fixture); (4) atomic Windows mutex and process-identity lock hardening across concurrent queue workers; and (5) deterministic 600-second idle GPU unloading. Finally, we document negative findings—including zero-reasoning formatting compliance trade-offs, single-slot FIFO queue waiting latency, and a transient 0 MiB free-VRAM log warning—and provide a complete reproduction protocol for engineering teams.

## 1. Introduction and Engineering Thesis

### 1.1 The Cloud Token vs. Local Slot Dilemma

Software engineering teams using agentic IDE environments (such as Cursor, Windsurf, or cloud CLI agents) incur substantial token consumption when frontier cloud models iteratively read files, synthesize patches, fail verification tests, and re-prompt within closed tool loops. Offloading bounded implementation slices to a local workstation GPU restricts the cloud orchestrator to task specification, diff verification, and test execution while moving high-volume code synthesis onto local hardware.

However, transitioning from cloud APIs to a single-workstation 16 GB GPU introduces two structural bottlenecks. First, a model whose quantized weights exceed 83 GB on disk cannot reside wholly in 16 GB of VRAM; achieving interactive throughput requires a sparse architecture where active parameters, attention caches, and hot weights fit within VRAM and host DRAM, while cold lookup structures are explicitly managed across NVMe storage and system memory. Second, a workstation GPU operating near its memory and compute limits supports a single active inference slot ( `slots = 1` ) at a 131,072-token context length. When multiple IDE sessions, background tasks, or agent lanes target the same local endpoint concurrently, uncoordinated requests exhaust memory, trigger premature timeouts, or cause operators to abandon the local queue and write changes directly with paid cloud tokens.

### 1.2 Engineering Thesis

A GPU's weight-storage capacity is not the same as the maximum size of a usable local model. A quantized sparse model, host-memory offloading, explicit storage placement, and a hybrid inference engine can support reliable local coding on a 16 GB GPU. However, obtaining tokens from a large model is only a fraction of the engineering problem: sustained usability requires strict context budgeting, non-reasoning client compaction, atomic resource ownership across a serialized job queue, deterministic idle resource release, and disciplined measurement.

## 2. Verified Artifact Identity and Provenance

All experiments in this study were executed using verified upstream software releases and cryptographically hashed local weight shards recorded on 3–4 November 2026 (IST) **[4]**.

- **Strata Inference Engine:** **https://github.com/Niko1221/Strata** (Terminal `0.2.9` / Control `0.1.55`, compiled for `CUDA 13`, `sm120`) **[1]**
- **Quantized Model Repository:** **https://huggingface.co/ISTA-DASLab/Qwen3.8-Flash-Next-GSQ-RCO-GGUF [2]**
- **Verified Model Repository Revision:** `ed59f92082b1e93c0e96d60a8b11aab089b52f09`
- **OpenCode Client Repository:** **https://github.com/anomalyco/opencode** (Version `1.19.5`) **[3]**
- **Local Native / API Alias:** `qwen3.8-flash-next-iq3_s` | **Client Model Identifier:** `strata/qwen3.8-flash-next-iq3_s`

| Artifact | Verified Size (Decimal Bytes) | Decimal GB | Verified SHA-256 Digest |
|---|---|---|---|
| **GGUF Shard 1** | 54,817,524,224 | 54.82 GB | `4c1eb2ceb4915e1192f4f386021897bde56a97f40a0bb78bb86465e0f7d2aca3` |
| **GGUF Shard 2** | 28,800,138,432 | 28.80 GB | `316b46f3a2dbd68c900f43136ab9449f9dcc3725dfd8c794847c204bc161e113` |
| **Total GGUF Weights** | **83,617,662,656** | **83.62 GB** | Two weight shards; runtime and MTP draft assets stored separately |

*Unit Convention & Baseline Distinction:* Disk file sizes use decimal units (bytes, GB = $10^9$ bytes), whereas memory allocations marked with binary units (MiB = $2^{20}$ bytes, GiB = $2^{30}$ bytes) reflect OS and GPU driver reporting. An 83.62 GB on-disk GGUF weight footprint does not represent an 83.62 GB VRAM allocation. In Section 5.1, the 83.62 GB sparse candidate ( `FlashNext` in `Strata` with `MTP` ) is compared against an existing local dense baseline: an approximately 11.8 GB on-disk, 26.9B-parameter dense `IQ3_S` model served via native `Ollama` .

## 3. System Architecture and Memory Hierarchy

### 3.1 Workstation Configuration Envelope

All measurements were recorded in an isolated portable environment on a single Windows workstation observed on 3–4 November 2026 (IST).

| Component or Parameter | Verified Value | Operational Role / Scope |
|---|---|---|
| **Operating Environment** | Windows (isolated portable lab root) | Isolated from system Python and existing model stores |
| **GPU Hardware** | NVIDIA RTX 5080 (16 GB; 16,303 MiB reported) | Primary compute and hot-weight / resident KV tier |
| **Host System Memory** | High-RAM host system (see Disclosure Box) | Holds offloaded model state and execution buffers |
| **Strata Terminal / Control** | `0.2.9` / `0.1.55` ( `CUDA 13` , `sm120` ) | Hybrid GPU/CPU/NVMe inference runtime |
| **Portable Python & Client** | Python `3.12.15` / OpenCode `1.19.5` | Isolated client state, SQLite DB, and configuration |
| **Storage Tiers** | Local NVMe SSD ( `<NVME_DIR>` ) | Active GGUF & direct `PLE` ; USB HDD for cold archive only |
| **Context & Concurrency** | `131,072` tokens, `1` inference slot | Single-slot serialized execution via OCF FIFO queue |

| Component or Parameter | Verified Value | Operational Role / Scope |
|---|---|---|
| **KV Cache Configuration** | `INT8` precision; `32,768` resident window | Balances long-context fidelity against 16 GB VRAM |
| **Coding Profile ( `deep` )** | High/Deep reasoning; `16,384` output cap | Default profile for all file-modifying coding batches |
| **Compaction Profile** | Same model via alias; `no-thinking` ; `4,096` cap | Prevents reasoning bloat during context summarization |
| **Vision / MTP Settings** | Vision disabled; `MTP` enabled (count `4` ) | Accelerates token generation; supplementary drafts logged |
| **PLE Placement Policy** | Explicit `direct` NVMe SSD access | Prevents automatic 26.8 GiB host-RAM preload spike |
| **VRAM Reserve & API** | `1,536 MiB` reserve; loopback HTTP API | Authenticated OpenAI-compatible API on `127.0.0.1:8081` |
| **Idle Resource Release** | `600` seconds without inference activity | Unloads model weights while keeping API listener online |

> **Workstation Host Memory Envelope (Optional Hardware Reproducibility Disclosure)**
>
> In the main narrative, host memory is described as a high-RAM host to preserve hardware privacy. For exact capacity reproduction: the test workstation is equipped with **96 GiB DDR4** system memory. Preflight startup guards check for approximately **75 GiB** available physical RAM and **85 GiB** OS commit headroom before loading the 128K profile. At a loaded 128K point sample, system RAM occupancy measured **72.2 GiB** alongside **15,074 MiB** GPU VRAM occupancy. Because host-memory channel geometry and PCIe interconnect bandwidth govern offloaded transfer rates, this configuration requires a comparable high-RAM host and will not run unmodified on standard 16 GB or 32 GB desktop PCs.

### 3.2 End-to-End Orchestration Topology (OCF V1)

The **OCF V1 (OpenCode FlashNext)** architecture decouples high-level task planning from token-intensive code generation across an explicit pipeline: (1) the human or cloud coding orchestrator writes a bounded coding-job specification ( `OCF_<ID>_PROMPT.md` ); (2) the FIFO queue and atomic ownership lock ( `state\ocf.lock` ) serialize slot access; (3) the isolated `OpenCode` client executes against the authenticated loopback `Strata` API ( `127.0.0.1:8081/v1` ); (4) the RTX 5080 inference engine coordinates hot execution with host memory (offloaded state) and local NVMe storage (model shards and direct `PLE` lookup assets); and (5) `OpenCode` performs scoped file edits and tool runs, returning a structured `RESULT` contract for independent diff review and test verification by the orchestrator.

### 3.3 Attribution Boundary and Memory Hierarchy Disambiguation

Adopters must distinguish upstream capabilities from the integration work performed in this study. Hybrid GPU/CPU/NVMe model execution, `MTP` speculative decoding, `INT8` KV caching, `PLE` storage flags, loopback API serving, and 600-second idle unloading are built into **Strata [1]**. Agentic tool execution, token accounting, prior-summary filtering, and compaction triggers belong to **OpenCode [3]**. The custom engineering contribution of **OCF V1 [4]** encompasses the multi-engine prompt/result contracts, PowerShell FIFO queue and lock runners, resource preflight guards, and the wire-level compaction request hook.

Furthermore, six distinct memory and storage structures operate simultaneously and must not be conflated as a single cache: (1) static GGUF weight shards on NVMe (`83.62 GB`); (2) active GPU VRAM allocation (up to `15,074 MiB` holding hot weights, buffers, and the `32,768`-token resident `INT8` KV window); (3) host DRAM offloaded state; (4) Per-Layer Embedding (`PLE`) lookup tables (`~28.8 GB` decimal / `~26.8 GiB` binary) accessed via `direct` NVMe reads; (5) the engine prompt prefix cache in memory (`98.09%` token reuse across turns); and (6) the client SQLite session database on disk.

## 4. Implementation and Optimization Methodology

### 4.1 Environment Isolation and Non-Destructive Preflight Guards

The experimental stack was installed in a self-contained directory root (`<LAB_ROOT>`) with isolated binaries, client state, and ports, preserving the workstation's existing `Ollama` installation. Launch checks refuse to load when another model owns GPU resources or when free VRAM (`14,000 MiB` check), physical RAM (`~75 GiB` check), or OS commit headroom (`~85 GiB` check) is insufficient, returning a retryable `HTTP 503` rather than terminating external processes.

### 4.2 Explicit Storage Residency (PLE Direct Mode)

During an early startup under an automatic placement policy, the engine selected full DRAM residency for `PLE` lookup tables, consuming an additional **28.8 GB decimal (26.8 GiB)** of host RAM and compressing desktop memory headroom. Pinning `PLE` placement explicitly to `direct` NVMe SSD access on `<NVME_DIR>` eliminated this 26.8 GiB RAM spike and made memory allocation reproducible across restarts. (Note: This optimization targets deterministic memory headroom rather than claiming SSD lookup is faster than DRAM; active lookup assets must remain on internal NVMe SSDs, never on high-latency USB HDDs.)

### 4.3 Context Budgeting and Non-Reasoning Client Compaction

An earlier client configuration triggered compaction near 60,000 accounted tokens while inheriting `High/Deep` reasoning and `16,384`-token output caps from the coding profile. Long reasoning traces bloated summary latency and shrank usable post-compaction context. The finalized 128K policy partitions the `131,072`-token slot as follows:

```
model context:          131072
coding input budget:    114688
coding output budget:    16384
compaction buffer:       32768
effective working bound: 81920  (114688 - 32768)
compaction output cap:     4096
compaction reasoning:     none
```

A proposed 90,000-token bound was rejected to preserve a full 32,768-token buffer for sudden multi-file tool bursts and the 16,384-token coding output budget. The compaction alias routes to the same model weights and endpoint without loading a second model. Because defining an empty `variants` map in `OpenCode 1.19.5` did not prevent inherited reasoning, our integration explicitly disabled inherited variants and pinned `reasoning: none` and `max_output_tokens: 4096` at the final outbound request hook (confirmed via wire capture).

Compaction uses a file-backed checkpoint prompt requiring ordered Markdown sections (`Goals`, `Constraints`, `Completed Work`, `Current State`, `Next Steps`) and an explicit end marker. A deterministic validator blocks continuation if any section is empty, out of order, duplicated, contains unbalanced code fences, lacks the end marker, or fails to finish with a natural `stop` reason. (Audit note: `OpenCode 1.19.5` already filtered older completed summary pairs from subsequent prompts; historical bloat stemmed from per-summary inherited reasoning, not infinite recursive summary accumulation.)

### 4.4 Shared Slot Ownership and FIFO Queue Hardening

Early sandbox testing (`S1`: `mergeRanges` passing 7/7 tests; `S2`: `clamp` passing 2/2 tests) identified and resolved four Windows runner issues: replacing Unicode em-dashes in `.ps1` files with ASCII; replacing `Start-Process -ArgumentList` (which caused exit code `-196608` on directory paths containing spaces) with `.NET ProcessStartInfo`, `Format-OcfWinArg` quoting, and `-f` file prompt attachment; computing before/after `git status --porcelain` deltas to ignore dirty workspace templates; and casting exit codes explicitly with timeout code `124`.

After reproducing four lock-ownership race conditions, the queue lock (`state\ocf.lock`) was hardened around six rules: (1) coordinate lock transitions with a shared Windows named mutex and atomic `FileMode.CreateNew` creation; (2) validate both integer `PID` and process creation timestamp to prevent PID-reuse collisions; (3) preserve live lock owners unconditionally regardless of age (never stealing a job simply because it exceeds two hours); (4) release only matching owner PID/job identities and preserve unreadable lock files; (5) keep `ocf-status` strictly read-only; and (6) reclaim only demonstrably dead owners while writing a terminal `failed` status if a worker exits with a job still marked `Status: queued`.

### 4.5 Idle Resource Release and Cold-Wake Verification

Configured with a 600-second idle timer, `Strata` releases GPU and host model memory after 10 minutes without inference requests while keeping the loopback HTTP server online. A controlled cold wake measured **30.515 seconds**. In passive runtime logs on 3–4 November 2026 IST, the last completion finished at `23:59:26 IST` and the final unload log entry occurred at `00:09:41 IST` (~616 seconds elapsed, reflecting the 600-second idle setting plus ~16 seconds of polling/teardown/logging overhead). Following unload, total system GPU memory usage dropped to **824 MiB** and later **612 MiB**, attributable to normal desktop/WDDM activity.

## 5. Empirical Evaluation and Results Ledger

### 5.1 Small Coding Screen: Dense Baseline vs. Sparse Candidate

Both deployed stacks were evaluated sequentially on six Python algorithm tasks comprising **505 deterministic executable test cases** (505 test cases across 6 problems) plus 3 exact-output/tool checks at an equal `65,536`-token context limit, `temperature = 0`, no reasoning, and a `2,048`-token output cap.

| Evaluation Measure | Dense Baseline (Ollama 26.9B IQ3_S) | Sparse Candidate (Strata FlashNext + MTP) |
|---|---|---|
| **Algorithm tasks passing every executable case** | 5 / 6 | **6 / 6** |
| **Deterministic executable test cases passed** | 504 / 505 | **505 / 505** |
| **Exact-output / supplied-evidence / tool checks** | 3 / 3 | **3 / 3** |
| **Instruction compliance: No-Import constraint** | **6 / 6** | 5 / 6 (imported `heapq` once) |
| **Instruction compliance: No-Markdown fences** | **4 / 6** | 1 / 6 (wrapped in fences) |
| **Median first-token latency (6 coding prompts)** | **0.312 s** | 1.058 s |
| **Aggregate generation duration (6 coding answers)** | **55.615 s** | 58.960 s |
| **Reported decode throughput (coding answers)** | ~58 tokens/s | **70.6 – 89.9 tokens/s** |
| **Long retrieval prompt length** | 60,071 tokens | 60,071 tokens |
| **Long retrieval first-token latency (TTFT)** | 33.735 s | **29.890 s** |
| **Long retrieval total duration** | 34.192 s | **30.479 s** |

The dense baseline timed out on a repeated-wildcard ( `*` ) stress case, whereas `FlashNext` used dynamic programming and passed all 505 cases. Short-prompt latency favored the dense baseline ( `0.312 s` vs. `1.058 s` ), and higher decode throughput ( `70.6–89.9 tokens/s` ) did not yield a shorter aggregate generation time ( `58.960 s` vs. `55.615 s` ) because answer lengths differed. On the 60,071-token retrieval prompt, `FlashNext` was faster in both TTFT and total duration. Grading stripped Markdown fences and permitted the harmless `heapq` import while recording instruction-compliance failures separately. An earlier unhardened trial also fabricated a warning about absent shell instructions, confirming that neither stack eliminates hallucinations.

### 5.2 Long-Context Operational Validation (64K and 128K)

Configured 64K ( `65,536` ) and 128K ( `131,072` ) profiles were validated across distinct operational fixtures: (1) actual `OpenCode` read/edit fixtures at 64K and 128K passed **108 / 108** start/middle/end marker checks; (2) a 128K non-thinking client fixture processed **112,321 input tokens** in **118.169 s** total duration; (3) a direct API retrieval request processed **124,003 input tokens** with a **60.816 s** first-token latency; and (4) a 128K Deep-reasoning client fixture processed **113,667 input tokens** in **168.036 s** total duration, passing **108 / 108** checks. Contexts above 128K (198K, 256K) and two concurrent 128K slots were not locally enabled.

### 5.3 Client Compaction Intervention Results

Using synthetic token usage counts to trigger the installed `OpenCode` compaction pathway followed by live local model summary and resumed-coding calls (after correcting a verifier whitelist defect prior to rerunning both arms), the request-hook intervention achieved the following results:

| Paired Compaction Fixture Measure | Before (Inherited Deep / 16K) | After (Pinned No-Thinking / 4K) | Verified Improvement |
|---|---|---|---|
| **Summary output tokens** | 1,898 | **575** | **−69.7% tokens** |
| **Compaction API elapsed time** | 69.282 s | **18.360 s** | **−73.5% duration** |
| **Independent resumed-function checks** | 55 / 55 | **55 / 55** | **100% parity** |

A subsequent hardening integration test produced **635 summary tokens** (natural stop, zero reasoning, **55 / 55** resumed checks), and an initial post-policy production summary generated **1,546 tokens** (natural stop, no error). Prior to optimization, one historical session snapshot showed **190,552 compaction tokens out of 441,151 client output tokens (43.2%)**, and a separate audit of **42 completed compactions** totaled **298,279 output tokens (~7,102 tokens/summary)**.

### 5.4 Runtime Throughput and Engine Telemetry Snapshots

Three separate engine log snapshots were recorded (cumulative retained logs and an active runtime sample):

| Telemetry Snapshot | Completed Requests | Generated Output Tokens | Weighted Decode Rate | Median & Additional Metrics |
|---|---|---|---|---|
| **Earlier Retained Engine Log** | 585 | 452,318 | **58.84 tokens/s** | Median 57.00 tok/s (range 16.4–89.9) |
| **Later Retained Engine Log** | 890 | 832,829 | **56.21 tokens/s** | Median 56.55 tok/s |
| **Active Runtime Passive Sample** | 48 | 27,707 | **54.25 tokens/s** | 510.77s decode, 86.43s prefill, 98.09% prompt reuse, 73.49% MTP acceptance |

Note that decode throughput excludes queue wait, tool execution, cold loading, and prefill. Prompt reuse ( `98.09%` ) is the token-accounting ratio `reused/prompt_total`. At a loaded point sample, GPU occupancy measured **15,074 MiB**, utilization **99%**, and core temperature **50 °C**.

### 5.5 Reliability Suite Validation and Operational Notes

All **4 / 4** reproduced lock failures, **10 / 10** shared-lock policy cases, and **10 / 10** checkpoint-format validation cases passed (correctly rejecting **5 / 5** previously accepted malformed inputs). In a two-worker concurrency sandbox, one queued job executed exactly once with zero leaked locks. Two operational anomalies are noted for transparency: (1) onestatus helper script retained an obsolete informational warning text mentioning a 2-hour age reclaim rule even though the deployed lock code preserves live owners unconditionally; and (2) an engine log warned of `0 MiB free VRAM` at the start of a request despite the `1,536 MiB` reserve setting, indicating that VRAM reserve should be monitored under heavy desktop multitasking.

### 5.6 Multi-Job FIFO Queue Dynamics and Orchestrator Token Offload

Across five tracked coding jobs—four serialized FIFO tasks ( `Job A` – `Job D` ) and one direct execution ( `Job E` )—operational telemetry recorded the following workflow distribution:

| Anonymized Job ID | Execution Path | Queue Wait | Run Duration | Local Tokens (In + Out) | Orchestrator Role |
|---|---|---|---|---|---|
| **Job A** | FIFO Queue (Slot 1) | ~0 min | ~14 min | ~118,000 | Enqueue + Verify |
| **Job B** | FIFO Queue (behind Job A) | ~7 min | ~22 min | ~144,000 | Background slice |
| **Job C** | FIFO Queue (behind Job B) | ~29 min | ~23 min | ~246,000 | Background slice |
| **Job D** | FIFO Queue (behind Job C) | **~42 min** | ~7 min | ~127,000 | Enqueue + Verify |
| **Job E** | Direct Execution | ~0 min | Direct run | ~137,000 | Enqueue + Verify |
| **Total (Jobs A–E)** | **4 FIFO + 1 Direct** | — | **~66m (A–D)** | **~772,000 – 773,000** | **~50,000 (Orchestrator total)** |

While the local stack processed ~772K–773K tokens (including 98.09% prefix reuse) and restricted cloud orchestrator usage to ~50K tokens, local token counts cannot be linearly subtracted from cloud billing due to caching, retry, and tokenizer differences. Furthermore, `Job D` waited ~42 minutes behind `Job B` and `Job C` before completing in ~7 minutes, demonstrating that single-slot FIFO serialization trades wall-clock queue wait for zero cloud token burn during slot contention.

## 6. Twelve-Stage Reproduction Protocol

1. **Verify Upstream Provenance & Hashes:** Confirm `Strata` (`0.2.9` / `0.1.55`), `OpenCode` (`1.19.5`), and `ISTA-DASLab/Qwen3.8-Flash-Next-GSQ-RCO-GGUF` revision `ed59f92082b1e93c0e96d60a8b11aab089b52f09`, verifying both GGUF SHA-256 digests.
2. **Create an Isolated Lab Root (`<LAB_ROOT>`):** Isolate Python `3.12.15`, client state, and configs from existing `Ollama` stores; verify NVMe capacity for weights, runtime assets, logs, and backups.
3. **Stage Active Assets on Local NVMe (`<NVME_DIR>`):** Download shards with a resumable tool onto internal NVMe SSD storage; never place active lookup tables on USB HDDs.
4. **Establish Conservative Single-Slot Baseline:** Configure `slots = 1`, `INT8` KV (`32,768` resident window), explicit `PLE = direct` SSD mode, and non-destructive `HTTP 503` preflight guards.
5. **Bind Loopback API & Smoke Test:** Bind authenticated API to `127.0.0.1:8081` using `<LOCAL_API_KEY>`; test health and a scoped disposable repo edit.
6. **Measure Baseline Telemetry by State:** Record decode, prefill, TTFT, total response time, and memory occupancy separately for cold and warm states alongside token counts.
7. **Validate 64K Before 128K:** Confirm retrieval and editing accuracy at `65,536` tokens before enabling `131,072` tokens.
8. **Deploy Pinned Compaction Alias & Checkpoint Guard:** Set the `81,920`-token trigger, disable inherited variants, pin `reasoning: none` and `max_tokens: 4096` at the request hook, and enable structural checkpoint validation.
9. **Add Atomic FIFO Ownership:** Deploy the Windows named mutex, `CreateNew` lock file, and `PID` + process start-time checks; validate with a mock executor before live LLM runs.
10. **Configure 600-Second Idle Unloading:** Verify natural idle weight release (~612–824 MiB residual GPU usage) and test cold reload (~30.5s) without polling inference endpoints.
11. **Execute Bounded OCF Jobs:** Write `.handoff/OCF_<ID>_PROMPT.md` with explicit `Allowed` and `Forbidden` files, enqueue, review `git diff`, and run independent tests.
12. **Package Sanitized Templates & Rollback Scripts:** Export configurations via an explicit allowlist with all private paths and keys parameterized.

## 7. Verified Evidence Audit & Sanitized Configuration Schemas

| ID | Technical Statement / Finding | Classification | Verification Basis |
|---|---|---|---|
| **V-01** | Two GGUF shards total 83,617,662,656 bytes (83.62 GB) with SHA-256 hashes `4c1eb2...` and `316b46...` at revision `ed59f92...` | Verified Measurement & Upstream Spec | Cryptographically hashed on test workstation by Arun C Michael (3–4 Nov 2026). |
| **V-02** | Pinning `PLE` to `direct` NVMe prevented an automatic 28.8 GB (26.8 GiB) host RAM allocation spike. | Verified Workstation Measurement | Observed during startup memory profiling; SSD-vs-RAM speed was not separately benchmarked. |
| **V-03** | Request-hook pinning ( `reasoning: none` , `4,096` cap) reduced compaction tokens by 69.7% (1,898 → 575) and latency from 69.282s to 18.360s (55/55 checks). | Verified Workstation Measurement | Paired fixture on OpenCode 1.19.5; wire-verified via loopback packet capture. |
| **V-04** | Sparse candidate passed 6/6 coding tasks (505/505 cases) at 70.6–89.9 tok/s vs. 5/6 (504/505) for 26.9B dense Ollama baseline at 64K context. | Verified Workstation Measurement | Sequential screen; sparse candidate had 5/6 no-import and 1/6 no-Markdown compliance. |
| **V-05** | 600s idle unload released model weights after ~616s down to 612–824 MiB system GPU usage; cold wake measured 30.515s. | Verified Measurement & Upstream Feature | Engine logs and WDDM GPU counters recorded 3–4 Nov 2026 IST. |
| **V-06** | 5-job wave processed ~772K–773K local tokens (waits 0, 7, 29, 42m) vs. ~50K orchestrator tokens; AC power and SSD TBW unmeasured. | Operational Workflow Metric | Production wave tracking; qualified for 98.09% prefix cache reuse and tokenizer differences. |

### Sanitized Public Manifest & Prompt Contract Templates

```
{
  "author": "Arun C Michael",
  "experiment_id": "ocf-v1-rtx5080-runtime-snapshot-01",
  "gpu_class": "NVIDIA RTX 5080 16 GB (16303 MiB reported)",
  "engine": "Strata 0.2.9 / 0.1.55 (CUDA 13, sm120)",
  "client": "OpenCode 1.19.5",
  "model_repository": "ISTA-DASLab/Qwen3.8-Flash-Next-GSQ-RCO-GGUF",
  "model_revision": "ed59f92082b1e93c0e96d60a8b11aab089b52f09",
  "local_model_alias": "strata/qwen3.8-flash-next-iq3_s",
  "context_limit_tokens": 131072,
  "inference_slots": 1,
  "kv_cache": { "type": "INT8", "resident_window": 32768 },
  "ple_placement": "direct_nvme",
  "compaction_policy": { "trigger_tokens": 81920, "buffer_tokens": 32768,
"output_cap": 4096, "reasoning": "none" },
  "active_sample": { "requests": 48, "output_tokens": 27707, "decode_seconds":
510.7722, "weighted_tok_per_sec": 54.25 }
}
```

## 8. References and Verified Artifact Provenance


1. **Strata Hybrid Inference Runtime:** Niko1221, *Strata Repository* (Terminal v0.2.9 / Control v0.1.55, CUDA 13, sm120 build), accessed and verified 3–4 November 2026. URL: **https://github.com/Niko1221/Strata**
2. **Quantized Sparse Model Artifacts:** ISTA-DASLab, *Qwen3.8-Flash-Next-GSQ-RCO-GGUF* (IQ3_S two-shard GGUF release, commit revision `ed59f92082b1e93c0e96d60a8b11aab089b52f09`), accessed and verified 3–4 November 2026. URL: **https://huggingface.co/ISTA-DASLab/Qwen3.8-Flash-Next-GSQ-RCO-GGUF**
3. **OpenCode Autonomous Coding Client:** AnomalyCo, *OpenCode Repository* (Release v1.19.5), accessed and verified 3–4 November 2026. URL: **https://github.com/anomalyco/opencode**
4. **Workstation Architecture, OCF V1 Protocol & Empirical Evaluation:** Arun C Michael, *Beyond VRAM Capacity: Single-Workstation Hardening, Compaction Control, and FIFO Queue Telemetry on an NVIDIA RTX 5080*, verified workstation engineering records, 3–4 November 2026 (IST).